

PROJECT 2: E-Commerce Inventory & Order Processing Engine

2.1 Project Overview

Build a console-based e-commerce back-end engine using any tool that manages product inventory, shopping carts, order processing, discount computation, and shipping-cost estimation. The system must make meaningful use of lists, stacks, queues, dictionaries, sorted linked lists, and sets, while demonstrating at least four GoF design patterns.

2.2 Learning Objectives

- Use generic collections (`List<T>`, `Dictionary<TKey, TValue>`, `HashSet<T>`, `Queue<T>`, `Stack<T>`) and custom linked / sorted linked lists.
- Apply the Factory Method, Decorator, Command, and Facade design patterns.
- Reason about trade-offs between data structures for different operations.
- Produce clean, testable code following SOLID principles.
- Document architecture decisions in a technical report.

2.3 Functional Requirements

- **Product Catalogue:** Store products in a `Dictionary<string, Product>` keyed by SKU. Support CRUD operations and keyword search.
- **Category Tags:** Each product has a `HashSet<string>` of tags (e.g., "electronics", "sale"). Use set operations (union, intersection) to power multi-tag filtering.
- **Shopping Cart:** Implement as a custom linked list of `CartItem` nodes. Support add, remove, update quantity, and calculate subtotal by traversing the list.
- **Order Queue:** Submitted orders enter a `Queue<Order>` for FIFO processing by the fulfilment module.
- **Undo / Redo for Cart:** Use two Stacks (undo stack, redo stack) of Command objects to allow customers to undo and redo cart modifications.
- **Price Sorted Display:** Maintain a sorted linked list of products by price for "browse by price" functionality.

- **Discount Decorator:** Wrap an IOrder object with one or more discount decorators — PercentageDiscount, FlatDiscount, BuyOneGetOneFree — that modify the total at each layer.
- **Shipping Calculator Facade:** A ShippingFacade hides the complexity of weight calculation, zone lookup, and carrier API stubs, exposing a single CalculateShipping(order) method.
- **Product Factory:** Use Factory Method to create product objects — PhysicalProduct, DigitalProduct, SubscriptionProduct — each with different shipping and tax logic.

2.4 Design Patterns Required

Pattern	Where Applied	Purpose
Factory Method	ProductFactory	Instantiate correct Product subtype from input.
Decorator	IOrder / DiscountDecorator	Layer multiple discounts onto an order dynamically.
Command	ICartCommand (AddItem, RemoveItem, UpdateQty)	Enable undo / redo for shopping-cart edits.
Facade	ShippingFacade	Simplify the shipping-cost calculation subsystem.

2.5 Marking Scheme (Total: 100 marks)

Assessment Criteria	Weight (%)	Max Marks
Correct implementation of six data structures	25	25
Correct implementation of four design patterns	25	25
Code quality (SOLID, naming, modularity)	10	10
Unit / integration tests (minimum 10 test cases)	10	10
UML class diagram and technical report	10	10
Git commit history and task-board evidence	10	10
Oral presentation and individual Q&A	10	10
Total	100	100